# DAY 13 – HASHING BASICS

Mastery of DSA in 50 Days

February 2026

## 1 Today's Focus

- Understanding the concept of Hashing
- What is a Hash Table and how it works
- Why hashing is so powerful
- Using hashing for frequency counting
- Understanding average-case time complexity benefits

## 2 What is Hashing?

Hashing is a technique that converts input data (key) into a fixed-size value (hash code) using a **hash function**. This hash code is used as an index to store/retrieve data quickly.

In simple words: Hashing = fast lookup, insert, delete using a magic mapping function.

## 3 What is a Hash Table?

A Hash Table is a data structure that stores data in **key-value** pairs.

```
Key       →   Value
"name"    →   "Dhilip"
"age"     →   25
"city"    →   "Chennai"
```

Internally:

- Hash function(key) → index in array
- Store value at that index

Python example: `dict`, Java: `HashMap`, C++: `unordered_map`

## 4 Why Do We Need Hashing?

Hashing helps to:

- Achieve near-constant time operations (O(1) average)
- Eliminate nested loops in many problems
- Solve frequency, existence, duplicate problems efficiently
- Trade a little extra space for huge time savings

## 5 Time Complexity of Hash Table Operations

**Worst case** happens due to collisions → handled by chaining or open addressing. Good hash function + load factor control → average O(1) in practice.

| Operation | Average Case | Worst Case |
|---|---|---|
| Insert | O(1) | O(n) |
| Search | O(1) | O(n) |
| Delete | O(1) | O(n) |

Table 1: Hash Table Time Complexity

## 6 Common Hashing Use Cases

- Frequency counting (characters, numbers)
- Finding duplicates / first non-repeating
- Checking existence fast
- Anagram / permutation validation
- Two-sum / pair sum variants
- Group by key (group anagrams)

## 7 Problem 1 – Count Frequency Using Hashing

**Problem Statement:** Given an array (or string), count frequency of each element.
**Example:**

```
Input:  [1, 2, 2, 3, 1, 1]
Output:
1 → 3
2 → 2
3 → 1
```

```python
from collections import Counter

def count_frequency(arr):
    freq = Counter(arr)
    for key, value in sorted(freq.items()):
        print(key, "   ", value)

# OR manual dict
def count_frequency_manual(arr):
    freq = {}
    for num in arr:
        freq[num] = freq.get(num, 0) + 1
    return freq

# Test
arr = [1, 2, 2, 3, 1, 1]
count_frequency(arr)
```

Time: O(n) Space: O(k) where k = unique elements

## 8 Problem 2 – Find Duplicates Using Hashing

**Problem Statement:** Find all elements that appear more than once in the array.
**Example:**

```
Input:  [1, 2, 3, 2, 4, 1]
Output: 1, 2
```

```python
def find_duplicates(arr):
    seen = set()
    duplicates = set()

    for num in arr:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)

    return sorted(duplicates)

# Test
arr = [1, 2, 3, 2, 4, 1]
print(find_duplicates(arr))  # [1, 2]
```

Time: O(n) Space: O(n)

## 9 Hashing vs Brute Force Comparison

| Approach<br>Best For | Time Complexity | Space Complexity |
|---|---|---|
| Brute Force<br>Very small n | $O(n^2)$ | O(1) |
| Hashing<br>Large n, fast lookup | O(n) | O(n) or O(k) |

Table 2: Hashing vs Brute Force

## 10 Key Learnings – Day 13

- Hashing trades space for incredible speed
- Average O(1) insert/search/delete is game-changing
- Frequency problems almost always solved best with hashing
- Python's `dict` and `Counter` make it very easy
- Understand when to use array vs hashmap (fixed range → array)

## 11 Interview Tips

- When you see "count", "frequency", "duplicate", "exists?" → immediately think hashing
- Ask: "What is the range of values?" → array possible if small
- Say: "Using a hashmap gives us O(n) time instead of $O(n^2)$"
- Mention collisions briefly if asked: "In practice, good hash functions keep it O(1) average"
- Always handle edge cases: empty input, all unique, all duplicates

## 12 Note to Learners

Hashing is one of the **most powerful and frequently used** concepts in DSA interviews.
It appears in:

- Almost every string problem
- Many array problems
- Two-sum, group anagrams, subarray sum equals k
- Top 100 most-liked LeetCode problems

Master it early — it will make hundreds of problems easier.

**End of Day 13**

**Next: Day 14 – Hashing Problems & Optimization**